Department of Engineering and Computer Science

Bachelor's Degree in
Computer Science

Final Dissertation

Structural Analysis of social networks in works in fan-fiction communities

Supervisor

Alberto Montresor

Student

Andrea Bragante

Academic Year 2021/2022

Acknowledgements

Questo lavoro è il capitolo conclusivo a questi tre anni di un percorso universitario che spero potrà proseguire portandomi in Europa.

Vorrei in primo luogo ringraziare la mia famiglia, quella naturale e quella allargata, composta non solo da parenti ma anche da amicizie che sempre mi hanno aiutano e spinto ad andare avanti.

Un ringraziamento va senza dubbio a David Lainado e Cristian Consonni, miei supervisori durante il tirocinio in Eurecat, che sono stati fondamentali per rendere l'intera esperienza interessante e proficua, considerando le competenze che ho potuto acquisire mettendo le mani sul campo; allo stesso modo ringrazio il mio supervisore prof. Montresor, il cui corso è stato senza dubbio una svolta per la mia passione neei confronti di questo campo.

Infine un grazie al mio collega Matteo Circa, con cui ho avuto il piacere di condividere opinioni, risultati ed esperienze durante lo svolgimento del tirocinio e la stesura della tesi.

Contents

Summary **2**

1 Introduction **4**
1.1 About fanfictions . . . 4
1.2 Archive of Our Own . . . 4

2 Data and Tools **5**
2.1 Tools used . . . 5
2.2 Structural Organisation of Data . . . 6

3 User-User graphs **8**
3.1 Building networks from user interactions . . . 8
3.2 Defining structural analysis metrics . . . 9
3.3 Defining centrality analysis metrics . . . 9
3.4 Works analysis . . . 10
3.4.1 Low clustering coefficient . . . 11
3.4.2 Works with non-star topology . . . 11
3.5 Correlation between metrics . . . 13

4 Comments trees **15**
4.1 Building comments trees from user interactions . . . 15
4.1.1 Resoning on orphan comments . . . 15
4.1.2 Threads . . . 16
4.1.3 Graph construction . . . 17
4.1.4 Anomalous works . . . 18
4.2 Defining tree analysis metrics . . . 19
4.3 Works analysis . . . 20
4.4 Works analysis aggregated per community . . . 22

5 Role of the user in different contexts **25**
5.1 Building the contributions dataframe . . . 25
5.1.1 Assumption on users . . . 25
5.1.2 Subset of users . . . 25
5.1.3 User behaviour metrics . . . 26
5.1.4 Building process . . . 26
5.2 Analysing contributions DataFrame . . . 27
5.3 Aggregated analysis per community . . . 28

Conclusions **30**

Summary

The entire entertainment sector has gone through big changes in the last few decades. The rise of the Internet, the appearance of online platforms for content fruition like Netflix and the almost unlimited availability of films, music, tv shows and video games, changed the way of approaching entertainment, letting new digital-based models arise. For what concerns the book sector, his new market represents a modern alternative, easily available and in some way more interesting and interactive, to the traditional market of paper books and ebooks: not by chance, the amount of young people who write, read and comment literature operas is constantly growing.

In these new models, the line between the role of content consumers and content producers becomes smaller and smaller, enhancing the figure of the *prosumer*. A prosumer is a user that does not strictly belong to a single category, but is instead flexible and moves in a spectrum in which can sometimes play the role of the *producer* and sometimes the one of the *consumer*. In this sector, *fan fiction* communities have carved out their own space. These are communities made of fans of some kind of opera, be it a comic, a film or a book; in these forums, users are prosumers, and hence can be creators of content or consumers.

There are plenty of advantages. From the consumer's point of view, the choice is incredibly high and can space between the most diverse topics and categories; moreover, it is in many cases free because works are amateur. From the producer's point of view instead, things get even more interesting. The feedback system integrated into these platforms allows users to express their opinion in different ways, at the point of possibly influencing the development of the work. The most interesting feedback mechanism is the comments system.

Each work can have more than one comment, and each comment can be a parent to others, allowing the development of conversations and message chains. These can contain compliments or personal opinions, but even more important are suggestions, criticism, ideas about the plot and so on; the author can take these exchanges and exploit them to develop their work following consumers' ideas and wishes, boosting synergy and cooperation.

The research project described in this thesis finds its basis on these assumptions. We asked ourselves if it was possible to gather data of some communities to study in deep the way prosumers behave, see how they interact online, represent these interactions through graphs and define metrics to analyse the general trends and most common patterns.

The research object is the web platform Archive of Our Own, one of the most important on the web, from which we retrieved data of works and comments from seven different communities: Marvel, Harry Potter, Sherlock Holmes, Lord of the Rings, Percy Jackson, Twilight and Warriors. On this dataset, we performed three different analyses, after the first step of cleaning data and organising data structures.

The first analysis, discussed in Chapter 3, is about users and the way they interact with each other.

We model through a graph the relationships between those who comment on a work, such that an edge between two users represents one or more reply comments between them; the comments posted as a reply to the work are considered as direct replies to the author, letting us introduce a node for them and allowing to analyse the relationships they hold with their community. We use metrics for graphs structural analysis, including clustering coefficient and reciprocity, and for users centrality analysis, like in-degree, out-degree and PageRank. What comes out is that there seems to be a persistent star topology in which the author is in the middle of the graph with pretty high centrality values. According to structural analyses, the distribution of the clustering coefficient related to the number of comments and commenters seems to be inverse, suggesting that by increasing the number of comments the interactions between users become less and less dense.

The second analysis, discussed in Chapter 4, is instead focused on comments.

We model with a tree structure the comments section of a work, with particular attention to

the parental relationships between them. We focus our attention on these to analyse the role of the author inside the nodes, how much users comment and how and with whom they prefer to interact. We introduce new metrics for this particular structure and define the concept of thread; we deal with the problem of orphan comments and anomalous works, reasoning about why it is important and the concept of bias in the analyses. What emerges, in this case, is that works tend to grow in depth and develop more if the author is active. Furthermore, we found that there are general trends valid among all the communities, even though with slight differences; thus we can think that factors like the type of content, the consumers' age range, or the fandom size have a more marginal role than we thought and that the users' behaviour is generally similar.

The third and last analysis, discussed in Chapter 5, is focused on the role of users.

Firstly, we reduce the dimensionality of our dataset for computational reasons, focusing only on a subset of authors, i.e. users who wrote at least one work. After that, we build a table in which, for each author, we insert the data related to their activities in two different contexts: when they are authors and when they are consumers; as we have seen, users in these platforms are prosumers, hence it is legit to think that an author also plays the role of the consumer. We then obtain information like how many comments a user writes, how much they are active in a community and how the role they play influences their activity; we use, apart from measures on the number of comments, also centrality metrics we discussed on Chapter 3 to identify the relevance of a user in a community. The results, even though not generalisable due to the smaller set of users, show how there is a consistent slice of authors on average more active in the interactions under their works than on other people's.

The final chapter consists of a brief summary of results found in previous analyses.

The entire research was performed as part of the wider European project Möbius, which is an initiative funded under the European Commission Horizon 2020 programme that aims to modernize the European book publishing industry by remodelling the traditional value chains and business models uncovering the prosumers potential and delivering new enriched media experiences.

1 Introduction

This chapter briefly introduces fanfictions and the platform we have chosen to analyse.

1.1 About fanfictions

A *fan fiction* is an amateur fictional entertainment work written by fans based on an existing opera; this opera could be a novel, a story, a movie, a comic and so on. The idea behind it is to have, as a fan, the chance to expand a story with own content.

Fanfictions knew an incredible rise in popularity with the spreading of the Web, encouraged by website like fanfiction.net or archive of our own, which are platforms that allow users to publish their works so that an entire community of fans can consume them, rate them and comment them.

The works would sometimes gain so much popularity that they get published. The most notorious cases are two: *50 shades of grey*, written by E. L. James, originally based on *Twilight*, got three books were published and movies were made from them; *After*, written by Anna Todd, originally a fan fiction about the musical group One Direction: five books published, three of which also became a movie.

1.2 Archive of Our Own

Archive of Our Own [1], shortened as AO3, is one of the most important platforms when talking about fan fiction communities. The website refers to itself as

> a fan-created, fan-run, nonprofit, noncommercial archive for transformative fanworks, like fanfiction, fanart, fan videos, and podfic.

In AO3 users can create works and post them on the platform, interact with communities of many fictional operas and use a system of tags to perform searches through collections of works; in addition, a user can have more than one *pseuds*, short for pseudonyms: users can post comments and works with a different pseudonyms, but they actually refer to the real username.

In addition, there is a system of feedback comprehensive of kudos, bookmarks and comments. Kudos is a way to show an author appreciation for their work: it is an ancient greek word meaning "glory" or "renown", and it acts as "likes" do in other popular social media platforms. Bookmarks are also a way to save works in a private collection in order to pick them up later. Comments system is even more complex and offers cues of analyses: users can post comments, replying to each other and building threads.

AO3 is the platform we have chosen to perform our analysis. We scraped data from seven different communities, chosen to be different in terms of size and original content type in order to guarantee fairness and diversity. The chosen communities are Marvel, Harry Potter, Percy Jackson, Lord of the Rings, Sherlock Holmes, Twilight and Warriors.

Below in Table 1.1 there is a small overview on fandoms, containing the number of works and comments:

[1] https://mobius-project.eu

Fandom	Number of works	Number of comments
Marvel	444804	10946620
Harry Potter	280310	6350061
Sherlock Holmes	124337	2749016
Lord of the Rings	24234	289388
Percy Jackson	19240	311157
Twilight	10974	141402
Warriors	2763	24472

Table 1.1: A few statistics about used communities

As we can see, not only they differ by size, but also by the number of comments they hold.

2 Data and Tools

This chapter goes through the process of collecting, cleaning and arranging data in a convenient way before the actual processing and analysis begin. We start by showing the tools we have used to perform our analysis and the way our data is structured; next, we talk about the actual pre-processing operation and show some general statistics about the data.

2.1 Tools used

Let us now do an overview of the tools we made use of to perform our research; our programming language of choice is Python [1], because of the great number of libraries available for data science.

- **Jupyter Notebook** [2] is a web interface for creating and sharing computational documents. Computational documents use cells as building blocks, and each cell can either contain text (written in Markdown) or code in a programming language of choice, Python in our case; this duality allows us to run snippets of code in cells, hence without having a single long script, and at the same time insert explanations and comments related.

- **Pandas** [3] is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool. The most important features, used throughout the whole research, is the *DataFrame* data structure, which allows to store and access data in a tabular way.

- **matplotlib** [4] is a comprehensive Python library for creating static, animated, and interactive visualisations in Python; these visualisations can be graphs, plots, bar charts and much more, and can be easily integrated in a Jupyter cell.

- **igraph** [5] is a collection of network analysis tools, available in different programming languages and usable with *matplotlib*. It is used in the research to represent graphs and trees in a convenient way.

- **PyMongo** [6] is a Python library containing tools to interact with MongoDB, which is the solution we use to store all the data scraped from AO3.

[1] https://python.org
[2] https://jupyter.org
[3] https://pandas.pydata.org
[4] https://matplotlib.org
[5] https://igraph.org/python/
[6] https://pymongo.readthedocs.io/en/stable

- **NumPy** [7] is a Python library that allows performing numerical calculations in a faster way; its array data structure implementation, called nparray, handles data more efficiently and is an extremely popular tool among data scientists.
- **SciPy** [8] is a Python library that provides algorithms for many classes of problems, for instance linear algebra, differential equations, statistics and much more.
- **statistics** is a library which contains easy way to perform mean and median calculations on arrays.

2.2 Structural Organisation of Data

Data scraping allowed us to collect not only a large amount of works metadata from the communities listed before, but also an even greater number of interactions between users about them.

The process of preparing data for the analyses is divided in two parts, one for the works and one for the comments. We start by just connecting to MongoDB using pymongo library and retrieving the data from the database. We create two DataFrames for each fandom, one holding information about works and one about comments, and easily fill the first one without too much elaboration: the only thing to worry about is replacing pseuds with the real username and ignoring works posted by orphan accounts, i.e. accounts which were deleted.

The resulting DataFrame holds the following fields:

- **id**, the id of the work in AO3;
- **authors**, a list of usernames who created the work: since the great majority of analysed works has only a single author, we assume the author is the first item of this list;
- **complete**, whether the work is completed or still in progress;
- **date_updated**, the date when the work was posted online;
- **hits**, the number of times it was read (or, at least, that the page was loaded);
- **language**, the language in which the work is written;
- **nbookmarks**, the number bookmarks the work got;
- **nchapters**, the number of chapters the work is made of;
- **ncomments**, the number of comments posted under the work;
- **nkudos**, the number of kudos the work got;
- **summary**, a brief summary of the work, made by the author;
- **tags**, a list of tags representing categories to which the work belongs;
- **title**, the title of the work;
- **words**, the number of words composing the work;
- **status**, whether the work is completed or still in progress;

The next step is to fill in the comments DataFrame. This time we have to perform some elaborations. For instance, we only want works with more than five comments or published after 2019; also, we want to remove comments posted by orphan accounts. The final step is to build comment threads, which will have a fundamental role later on. To achieve this, we iterate on comments and add two columns *reply_to* and *parent_id*.

Let us now see the resulting DataFrame:

[7] https://numpy.org
[8] https://scipy.org

- **chapter**, the chapter of the work in which the comment was posted;
- **datetime**, the date and time when the comment was posted;
- **parent_id**, the id of the parent comment; has value -1 if there is no parent;
- **text**, the actual text of the comment;
- **work_id**, the id of the work to which the comment belongs;
- **comment_id**, the id of the comment;
- **comment_author**, the comment author;
- **text_len**, the length of the text;
- **work_author**, the username of the work author;
- **reply_to**, the username of the parent comment;

Other relevant DataFrames

Although these are the most used DataFrames in the whole research, they are not the only ones.

For instance, it is useful to keep track of users, authors and commenters, each in a different DataFrame containing general information about them. Another important DataFrame is called eligible works and contains works with more than twenty comments, which will get used in Chapter 3.

3 User-User graphs

This chapter goes through the process of building and analysing the social network in the comments section for each work of each community; the graph we are building has users as nodes and we want to model the interactions between users on a comment level.

The goal we are pursuing with this analysis is to understand what structural properties lie underneath these networks; furthermore, we are interested in visualising their graphical representations to investigate on whether there is a recurring topology or if the way users interact through comments can result in different shapes.

3.1 Building networks from user interactions

Before starting to represent the network related to one specific work, we first have to identify the actors playing and their relationships, and map their role into nodes and edges. For the next analyses we are just focusing on how and how much users interact with each others, not minding too much about their role in the work: thus, we model comment authors as nodes and the number of interactions between them as edges.

We start by building a list of vertices, i.e. the users who posted at least one comment, for each work of each community. We proceed calculating the set of unique users who commented a specific work, convert it into a list and then store it into a *Pandas* DataFrame. It contains one row for each user containing their username, referred to as *username*, and an integer uniquely identifying the vertex called *vertex_id*; while the former is found in the comments DataFrame, the latter is generated iterating the list of users and taking the index in the list.

We then move onto building a list of edges, i.e. pairs of vertices representing an interaction between two users, for each work of each community; every interaction is basically a reply to one comment, meaning that some user A replied to some user B's comment, creating a link. The procedure is similar as before: we iterate on comments, group them by work and store them into another *Pandas* DataFrame. We then remove self comments, since they are not relevant to the analysis, and aggregate to form an edge by exploiting fields comment_author and reply_to mentioned above; we create a new column called *replies*, which contains the number of different comments constituting that edge, which will actually be used later as weight parameter.

Finally, we can put together our nodes and edges to create a *weighted directed user-user graph*. To achieve this, we use *igraph*, which simplifies the procedure allowing to build a graph starting from DataFrames; from here, we can tinker our graph to remove weights and obtain an *unweighted directed user-user graph*.

Reasoning about weight

When building edges, it is questionable whether to keep track of the weight of an edge is useful or not: in a way, it measures the strength of the relationship between two users. For the sake of the next analyses, it is useless since we are only focusing on the undirected graph, but it might come in handy if we were analysing the whole community instead of a single work.

When focusing on one single work's interactions network, it is indeed necessary to consider all users who commented and hence not apply restrictions on nodes; on the other hand, when considering a whole community, there may be users more relevant than others, in particular those who had a higher number of interactions in the network.

Hence, for this kind of analysis, the weight is not going to be used as threshold not to consider those bland interactions from negligible users.

3.2 Defining structural analysis metrics

Now let us define the first set of metrics we are using, *structural metrics*. They are calculated on a network level, and therefore we can conveniently visualise the result in a table for each work of each community. As said before, the goal of this analysis is mainly to inspect the networks, understand how they are connected and rise hypotheses on the reasons behind it; for instance, how much do user interact? Do they communicate with each others or only with the author?

Let us move onto the metrics:

- **Number of nodes**, which is the number of distinct users who left at least one comment under the work.
- **Number of edges**, which is the number of interactions between pairs of users under the work. Each one of these interactions could actually be constituted by a variable number of comments, so without looking at the weight we have no actual idea of how many comments a user replied to another one with.

  It is important to note that some comments are not meant to be replies to some others, but rather they are directly posted under the work: we map this interaction as between a user and the author.
- **Clustering coefficient**, which is a measure of the degree to which nodes in a graph tend to be grouped together. There are two different version of this metric, a local one and a global one (which we are using here):
  - The *local* measure is node-centric, and focuses on how much one node's neighbours are connected with each other, implicitly meaning how relevant is that node in its cluster (in other words, how many nodes would get disconnected if the node is removed?).
  - The *global* measure is instead based on triplets of nodes. A triplet is a set of three nodes: it can be either open, if the nodes are connected by two edges, or closed, if they are connected by three edges. The global clustering coefficient is given by the number of closed triplets over the total number of triplets in the network; it basically tells us how much the users interact with each others.
- **Reciprocity**, which is a measure of the attitude of nodes to be mutually linked, in a nodes-pair-wise fashion. For two nodes A and B, to be mutually linked means that both have an edge direct to the other one; in other words, in a directed unweighted graph we could easily collapse the two edges and replace them with an indirect one with no loss of information.

3.3 Defining centrality analysis metrics

Now let us define the other set of metrics we use, *centrality metrics*. Unlike structural metrics, these are related to every single node inside a network, and hence we cannot visualise the result for one work as a whole, but instead we have the values for each user in the network. Since the number of commenters is pretty variable among the communities, it would only be messy to see the whole list of users with their metrics.

For this reason, we have decided to focus on the most interesting results and sum them up in three rows, containing:

- the metrics applied to the author of the work;
- an average of the metrics values for each user;
- an average of the metrics values for each user, but without considering the author in the counting.

Before taking a look at the metrics, let us recap some graph theory: the degree of a node in a graph is the number of connections it has with other nodes, represented in our case as interactions between users.

We can now move on and define:

- **In-degree**, which is the number of users who replied, with one or more comments, to a certain user.
- **Out-degree**, which is the number of users whom a certain user replied, with one or more comments.
- **PageRank**, which is a measure of the relative importance of a certain user within the network; the measure is obtained through the PageRank algorithm, developed by Stanford University and still used as part of Google page-ranking algorithm, and it considers not only the amount of edges pointing to a user, but also the importance of the users from which those edges start.
- **Pagerank rank**, which is a more convenient way to interpreter the PageRank value; we used the function `rankdata()` from the library NumPy to rank each value in the PageRank array (e.g. given an array [1 8 5 7 9], rankdata returns an output of [1. 4. 2. 3. 5.]).

3.4 Works analysis

Let us now choose a small set of works so we can inspect them and dig into visualisations and metrics.

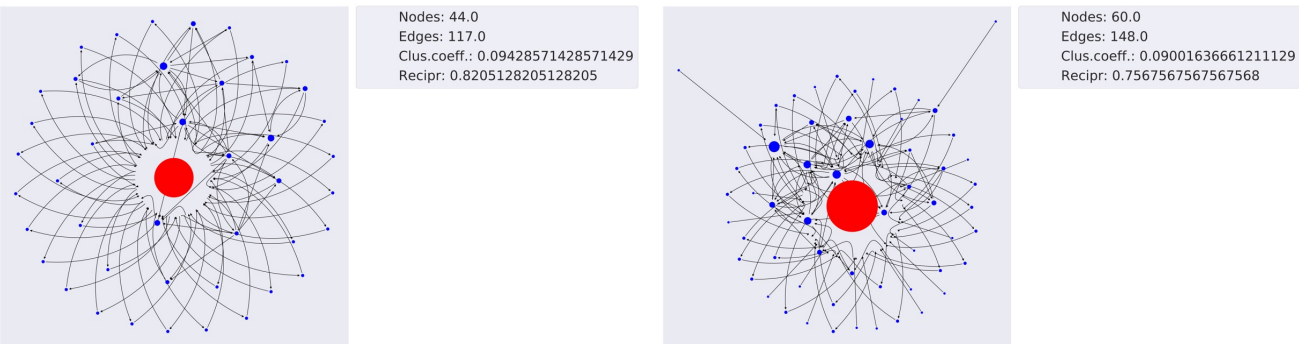
The works we have chosen can be found online on the website, replacing the work id parameter in the following URL: https://archiveofourown.org/works/id. The reason why we have chosen specifically these works is to focus on works with very high or low metric values, in order to also have a visual idea on how that metric gets reflected on the graph. We graphically visualise networks to see at a glance the most relevant features, such as how dense are the connections and which users are the most active.

Each user is represented by a node, which can be either coloured in blue or red whether they are normal users, i.e. commenters, or the work author; furthermore, nodes are bigger proportionally to their in-degree.

Connections are represented by arrows pointing to the destination node. On the top right corner we can see a small panel containing work's structural metrics and general work information to be conveniently looked up on the site or other DataFrames.

High clustering coefficient

The following works in Figure 3.1 are some of those with a high clustering coefficient value among all communities, and are examples of the almost ubiquitous star topology: this implies that the work author is the biggest node in the middle of the network and is connected to the other users either by out-going or in-going edges.



(a) "A Guy Could Do Worse"
Marvel, ID=10426608

(b) "The Return"
Harry Potter, ID=11675088

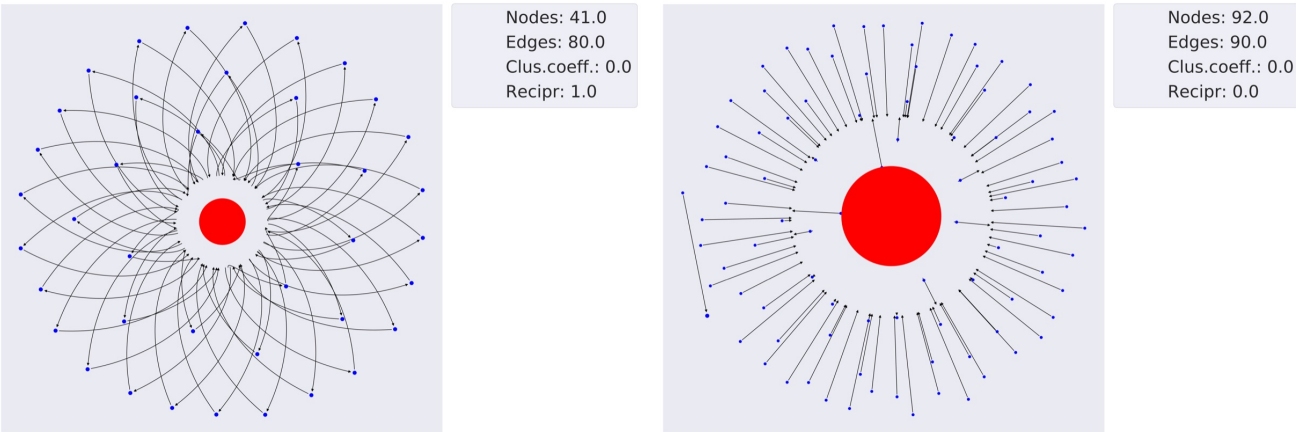
Figure 3.1: High clustering coefficient

Since the clustering coefficient seems to be inversely correlated to the number of commenters (i.e. the more the commenters, the smallest the value) we decided to choose two works with also a high number of distinct users (more than 40 in this case) to avoid trivial graphs.

It is particularly interesting to note the clustering coefficient value among works: although these are some of the works with a high value, it still is drastically low, i.e. it is never above 0.1; this could be an indicator of how users tend to prefer to interact with the work author rather than with each other, as also suggested by the star topology itself.

3.4.1 Low clustering coefficient

The following works in Figure 3.2 are some of those with a very low clustering coefficient value, which are the majority among all the communities. Furthermore, we have decided to pick one with a very high reciprocity value and the other with a very small one, to see how they differ.



(a) "Sunday Evening 6 p.m."
Sherlock Holmes, ID=25293388

(b) "Fairy Tales and Clockwork Hearts"
Marvel, ID=1830616

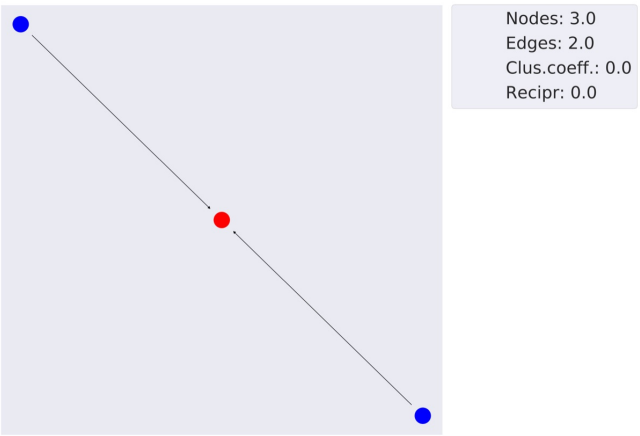
Figure 3.2: Low clustering coefficient

There are a few interesting things to observe here. Both the works have a clustering coefficient equal to zero, meaning that users interact only with the work author; also, there is a remarkable difference between the number of nodes, and this led us to think about the influence that the number of commenters (and comments) has on the other metrics: we cover this later on.

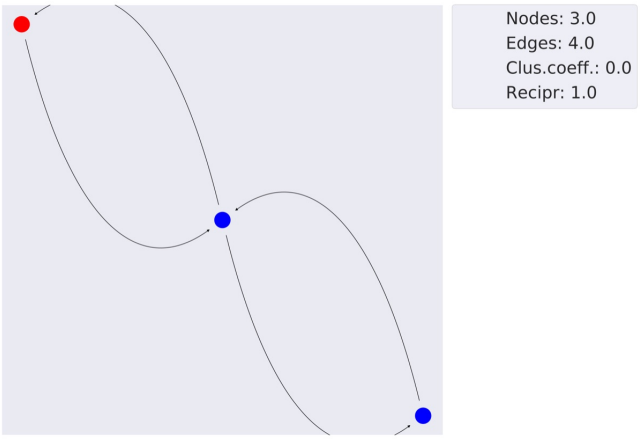
Also, the work in Figure 3.2a has the highest reciprocity value possible, while the one in Figure 3.2b has the lowest possible. Hence, in the first on the left we can see how each blue node has a double connection with the work author, one in-going and one out-going, and thus the number of edges is exactly two times the number of nodes minus one (the author); on the other hand, in the one on the right the author posts no comment, and we can clearly see it because there are arrows only pointing towards them.

3.4.2 Works with non-star topology

Of course every work is a small world on its own, and each community that develops around it is different from the others. As we can see below, there are also some non author-centric topologies; it is important to note though that these works have a small number of nodes and edges and this factor may play a role in the resulting shape.



(a) "5 Times Someone Thought They Were A Couple, And 1 Time They Actually Were" Sherlock Holmes, ID=779969



(b) "Dr. Choś Unexpected Date: Confessions" Marvel, ID=16150034

Figure 3.3: Anomalous works

As we can see, the first work, shown in Figure 3.3a has a segment structure and less than three nodes, one of which is the author. The clustering coefficient is null, but in some cases it might be even non-computable: if a graph had less than three nodes, the clustering coefficient would not be a number since it gets computed over triplets of them.

The work in Figure 3.3b has an uncommon shape but comprehensible metrics: again, the clustering coefficient is null, but this time each interaction has a mirror one, raising reciprocity to 1.

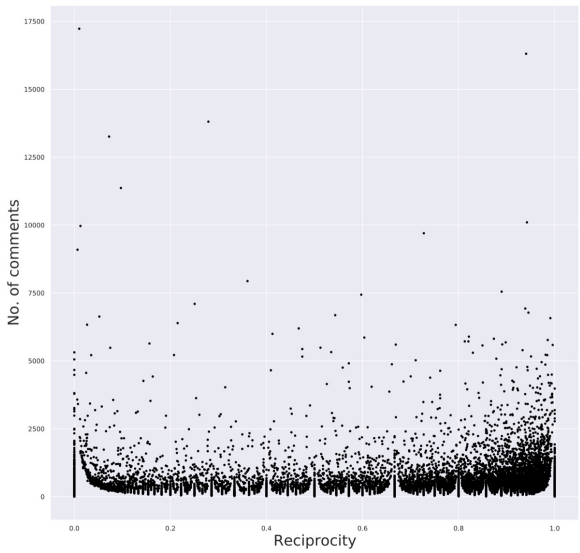
3.5 Correlation between metrics

Finally, we wanted to investigate correlations between some of these metrics, in particular focusing on clustering coefficient and reciprocity. We plotted four graphs using *matplotlib*, each one focused on a different relationship between parameters.

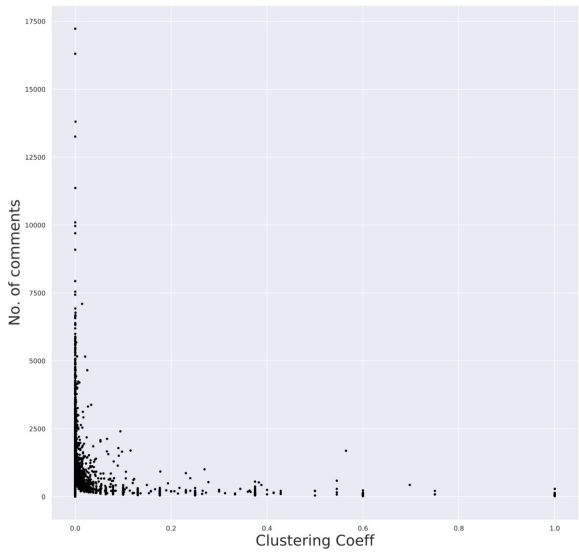
Let us take a closer look to see what we discovered.

By number of comments

We first inspect the graphs having on the x-axis the values of clustering coefficient and reciprocity and on the y-axis the number of comments.



(a) Correlation between number of comments and clustering coefficient



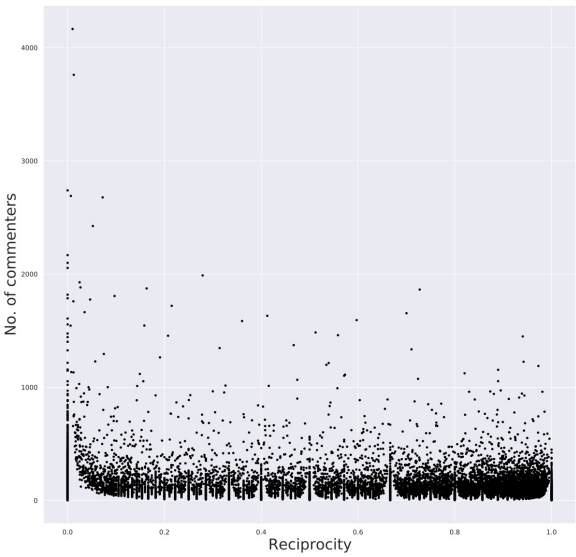
(b) Correlation between number of comments and reciprocity

Figure 3.4: Correlation between number of comments and metrics

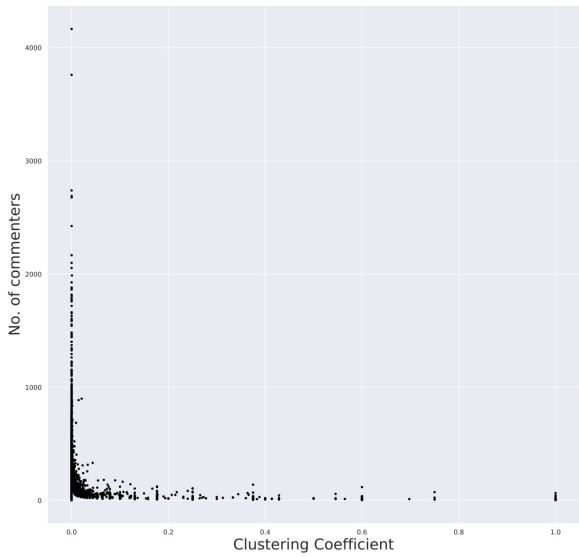
For what concerns clustering coefficient, we can look at Figure 3.4a and catch a glimpse of an inverse correlation: the highest the number of comments, the smallest the clustering coefficient, being consistently near 0. The concentration of works is very high near the plane origin, i.e. where both the number of comments and clustering coefficient are small. For what concerns reciprocity instead, we can look at Figure 3.4b and note how points are quite equally distributed on the plane, with only a slight increase in comments when reciprocity is higher and several lone points below the 10000 comments threshold.

By number of commenters

We proceed inspecting the graphs that still have on the x-axis the values of clustering coefficient and reciprocity, but on the y-axis the number of commenters.



(a) Correlation between number of commenters and clustering coefficient



(b) Correlation between number of commenters and reciprocity

Figure 3.5: Correlation between number of commenters and metrics

The outcomes are actually pretty similar to the previous analysis. We can see in Figure 3.5a a more sharp line where the clustering coefficient is equal to zero, but the inverse correlation is still preserved as before. In Figure 3.5b, reciprocity seems to be a bit more regular.

4 Comments trees

This chapter goes through the process of building and analysing the intrinsic tree structure of the comments section of each work of each community.

The goal we are pursuing with this analysis is, similarly to Chapter 3, to understand if we can extrapolate properties from these trees and the way they are structured to form some kind of pattern that could lead to formulate hypotheses on them. As before, we can graphically visualise the graphs in order to have a concrete representation of what we read in the tables. We will also discuss the problem of orphan comments, the issues with anomalous works and finally show two analyses: the first one is performed on two works, while the second one is performed community-wise and aggregates results found.

While in the previous chapter we wanted to model the relationships between users and the way they interact, here we want to focus only on comments, hence we build a graph which highlights the tree structure: every node is a comment and the interactions are modeled as replies to some other comment.

4.1 Building comments trees from user interactions

We define a procedure to generate a tree starting from its workid and fandom. As before, let us first specify some basics.

The tree we are going to construct is a generic one. There is one root node which does not represent any actual comment, but is used as ancestor of all other comments: it represents the work itself, so in a way has the same goal as the author node had on the previous chapter. We model each comment of a work as a node: each node has a parent and may have zero or more children. Nodes are linked together by *reply_to relations*, i.e. a node B is child of a node A if B is a reply comment to A. Each node has a set of attributes used both in the process of metrics calculation and graph construction; let us briefly discuss them (noting how the root node has specific fixed values for convenience reasons):

- **id**, the id of the node in the graph; the root node has `id=0` because it is the first node.
- `comment_id`, the `id` of the comment the node represents; the root node has `comment_id=-1` because there is not an actual parent comment.
- **parent**, the `id` of the parent node in the graph; the root node has `parent=-1` because it has no parent.
- **author**, the username of the comment author; the root node has `author='work_author'`.
- `is_leaf`, a boolean value representing whether the node is a leaf or not; all nodes have `is_leaf` attribute initially set to `False`.
- `thread_id`, a list of integers containing the ids of the threads the node belongs to; all nodes have `thread_id` initially set to an empty list.
- **children**, a list of integers containing the ids of the node's children; all nodes have `children` initially set to an empty list.

4.1.1 Resoning on orphan comments

At this stage, we faced a subtle problem: orphan comments. Not all the comments of a work are in the DataFrame: deleted comments, for instance, get removed on the clean-up stage as we have seen in Chapter 2. This means that the DataFrame containing all comments will have some rows whose parent id cannot be found: these are what we call *Orphan Comments*.

Of course this is just on the shallow level: each orphan comment could be parent of an entire subtree of children comments, which are in turn orphans in a way. The problem with this is that we have no way to know where these nodes were in the tree. For instance, one could have been the first node originating a thread, or maybe even a subtree of nodes, carrying with it plenty of children; one could instead have been just a leave, and hence be quite harmless in the big picture; finally, one could have been in the middle of the tree and lead to a bifurcation, i.e. a node with two distinct replies that correspond to two different subtrees.

Solution ideas

There were three different solutions which we came up with to deal with this issue.

First, we could consider orphan nodes as smaller separate trees: we could represent the missing parent node creating a new dummy node and using it as parent for its children. This is actually the way we treat orphans during the construction of the tree. Ideally, we could then run the same metrics on these smaller trees and tinker with them in some way not to loose the information those nodes carry.

A similar idea could be to consider all orphan nodes as a tree on its own: every children could be linked to a new dummy root node and, as before, we could run the metrics on this other tree. Of course the results would probably not be indicative of the actual structure of the original tree, because the nodes could be just an agglomerate of random comments.

The third solution, which is also the one we decided to implement, is to remove them all. If the number of orphans is not relevant and their analysis is useless, it would be convenient and computationally easier to just get rid of them.

The point is that no matter which solution we implement, the metrics calculated on the tree will be biased in some way, because the tree is not the actual one but an engineered version; hence, let us try and compute how relevant is this problem overall and then decide if we can safely remove these nodes.

Orphan comments statistics

To have an idea of the impact of orphan comments on the total, we iterate on them and on all of their children to find out whether it is safe to remove them or not. What we end up with is the number of orphan comments for each fandom. We build a DataFrame and visualise data it in tabular way: we compare, for each community, the number of orphan comments with the total number of comments.

The result is the following:

	Total Comments	Total Orphans	% Orphans
Marvel	10946620	104711	0.956560
Harry Potter	6350061	37682	0.593412
Sherlock Holmes	2749016	32303	1.175075
The Lord of the Rings	289388	1580	0.545980
Percy Jackson	311157	3166	1.017493
Twilight	141402	542	0.383304
Warriors	24472	370	1.511932

Figure 4.1: Table showing the percentage of orphan comments on the total

As it turns out, the numbers in Figure 4.1 are very small: they do not even get up to 2%. In fact, if we tried to plot total comments and orphan comments with two bar charts, the orphans bar would be so small we could not even notice it. Thus, we can safely remove all these nodes.

4.1.2 Threads

Another concept to introduce before diving into the tree construction algorithm is the concept of *thread*.

We define a thread as a sequence of comments with parental hierarchy, i.e. every node in a thread is either the child of another node in the same thread or directly the child of the root node; a thread is made of at least two comments: the *generating node*, which is a node of level one (i.e. directly connected to the root), and at least one reply to it.

A thread is defined starting from a leaf node and then following the path back to the root (which is not included though because it would be useless, since it would belong to every thread in the tree). This way we can have a precise idea of different conversations under a work, including potential bifurcations.

Another way to think about threads could have been more conversation-wise, for instance modelling the entire subtree with root in a level-one node as a thread and thinking of it as the conversation generated from a single direct comment; however, this way we may have troubles considering some metrics, for example the depth of a thread or the percentage of author replies, hence we stuck to this definition.

4.1.3 Graph construction

We start by creating a list of vertices for the new graph, mapping each comment of a work to an index, starting from 0. We can then create the graph from the list of vertices and initialize seven empty lists, one for each attribute listed before to fill out in the next step.

We can now perform some pre-computation before starting to add the edges. First, we compute the parent list iterating through the `parent_id` column in the DataFrame. We want a list, ordered as the ids list (for correspondence), where each element is the parent id of the node at that index. If the row does not contain a valid `parent_id`, we create a new vertex and assign it as parent.

Then we compute the id list, just by using the vertices list made before, the `comment_id` list, by using the values on the `comment_id` column in the DataFrame, and the author list, again by using the values on the author column in the DataFrame. Finally there are `is_leaf`, `thread_id` and `children`: they are computed during the iterations, hence we can initialise them as True for `is_leaf` and as empty lists for the other two.

To continue, we can just iterate through each comment and create an edge between the node and its parent; in addition, we want to assign a `False` value to the attribute `is_leaf` of the parent node, since it clearly has a child.

The next step is to remove all orphan comments. They currently are included in the graph, but since they are not attached to the main root node but instead on a different one, we can actually consider them as connected components. This is pretty convenient, because we can just forget about them by replacing the graph reference with the biggest connected component in the graph, by performing `graph.clusters().giant()`.

The last step is about computing the thread attributes. Basically, we iterate on each leaf and assign a new thread id to that leaf; then we walk back to the root and append that thread id to each node we come across in our path. After that, we have finished our tree construction and can move on to our analyses.

The result can be seen below in Figure 4.2 with the tree generated by a random work.

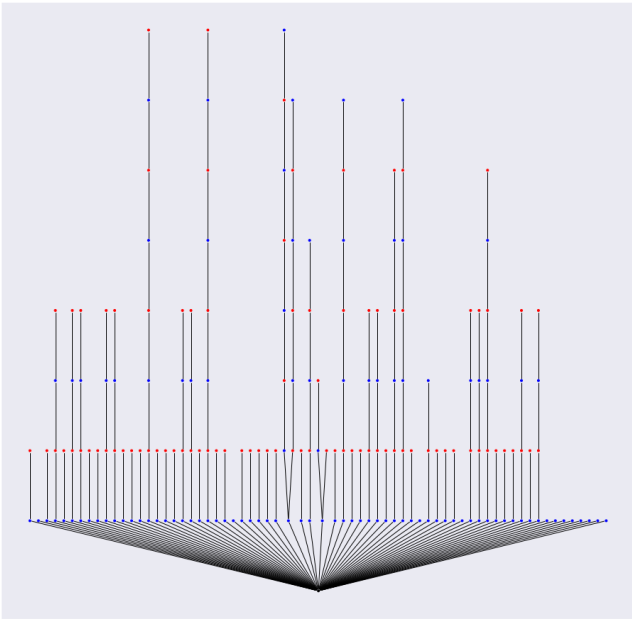


Figure 4.2: Tony Stark... not recommended
Marvel, ID=3521582

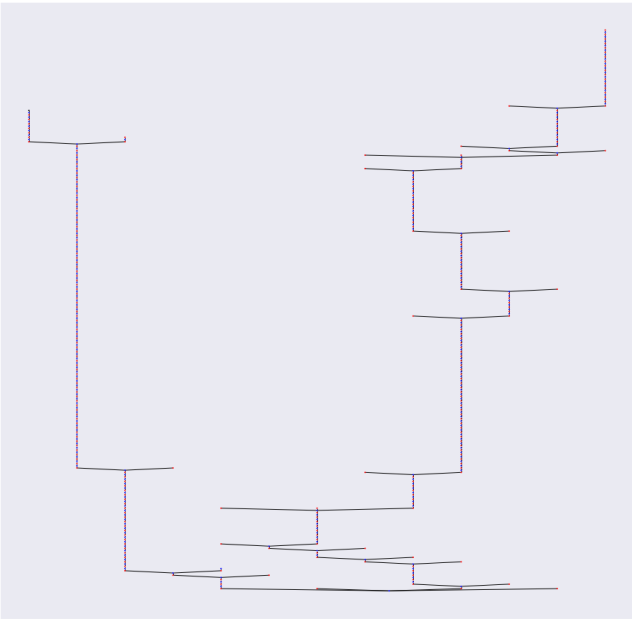
Similarly to the previous chapter, the red nodes are comments made by the work author, while the blue ones are made by some generic user. We can see how the root is coloured in black to distinct it from a normal node. We can also see some examples of bifurcations, i.e. nodes from which more than one branch starts, and the difference between single comments and threads: single comments, also called generating nodes, are those level-one comments which do not originate any thread and are just a single direct reply to the work; threads, on the other hand, are more complex structures and also the more interesting ones.

As in the previous chapter graphs had an almost-ubiquitous star topology, there is a recurring feature here as well, i.e. the red/blue nodes alternation.

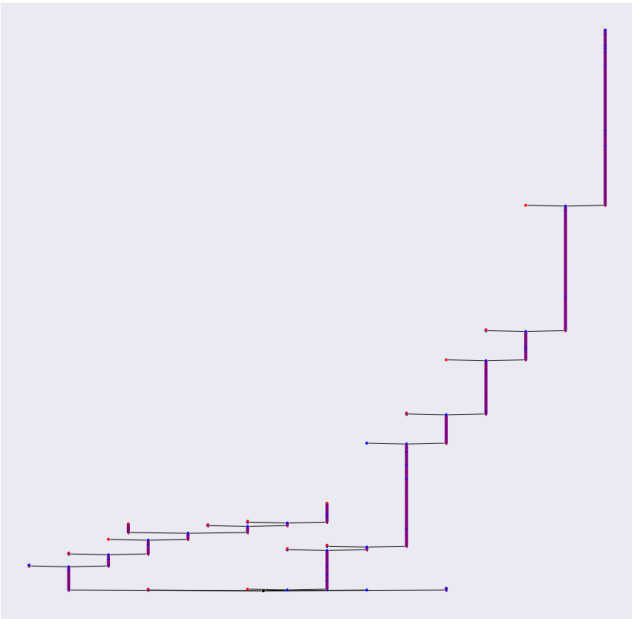
4.1.4 Anomalous works

Before showing how we analysed the trees with our metrics, let us briefly talk about what we call *anomalous works*.

While studying trends and patterns in the trees, we noticed that some woks were different from the ones we had seen before, having some metrics values pretty far from the mean. We can see a couple of examples below.



(a) Thunder Smiles Warriors, ID=3064727 work



(b) New Kin brings new Challenges Warriors, ID=18630685

Figure 4.3: Anomalous works

The work we are going to focus first is the one in Figure 4.3a: The first thing to note is the depth of the tree and the total number of comments, which as we can see is very high: to be precise, this tree is made of 492 comments, making it one of the top five most commented work of the entire Warriors community.

If we were to read its analysis, we could see that there are only three users posting comments: ScarletLeaf (Guest User), ScarletMarieLeaf and Wolffang009; the first one actually only made one comment, and since it has almost the same username as the second one but is labeled as guest user, it probably is the same person who once posted with a guest account by mistake. This means there are only two people interacting, one of which is the user. The resulting tree has two big branches, one of which is basically just a chain of replies between user and author.

Taking a look at the actual work on the website, it turns out that the two were roleplaying, i.e. expanding the fan fiction by interpreting two different characters and creating a new story comment after comment.

Let us now focus on the other example in Figure 4.3b. This time, we have a work with a lot more comments, but still a reply chain made by only two users: if we went to check on the website, we would find a chain made by what seems to be a conversations between a user and the author.

The concern with this kind of works is that they could bias the overall metrics, pushing the mean towards not quite indicative values. However, we decided not to deal with this problem for two main reasons:

- It would not be correct to remove works that do not follow a general trend because we would be a manipulation of trends;
- It does not seem to be a statistically significant problem.

4.2 Defining tree analysis metrics

We can now proceed by defining the various metrics we are going to apply on comments trees. Since they are more than a dozen, they are divided into four categories in order to keep it clear:

- Tree overview
  - **Number of comments**, the total number of comments under the work;

  - **Tree Depth**, the depth of the tree's deepest leaf;
  - **Tree Breadth**, the number of nodes in the widest layer;
  - **Widest Layer**, the level with the greatest number of comments;
  - **% Single Comments**, the percentage of level-one comments over the total.

- Author role
  - **Is Author the most active user?**, a boolean indicating whether the work author is the one who commented the most;
  - **% of Author Comments?**, the percentage of comments made by the work author over the total;
  - **% of Author-replied comments?**, the percentage of other users' comments to which the author replied.

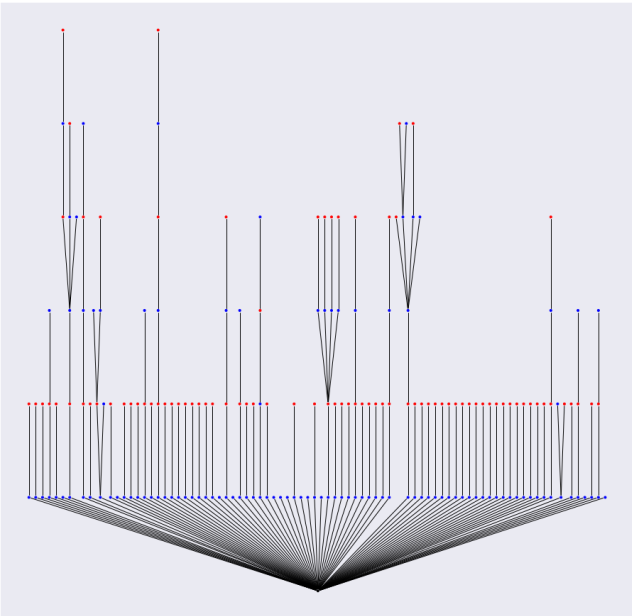
- Commenters role
  - **Number of distinct users**, the number of distinct commenters under the work, including the author;
  - **Average number of comments per user**, the average number of comments per user;
  - **Average number of comments per user (no author)**, the average number of comments per user excluding the author;

- Threads-related
  - **Average thread length**, the average number of comments that form a thread;
  - **% of threads long 1 or 2**, the percentage of threads with one or two comments, even though a thread made by one comments is not properly a thread;
  - **% of threads longer than 3**, the percentage of threads made by more than 3 comments;
  - **% of threads longer than 5**, the percentage of threads made by more than 5 comments;
  - **% of threads longer than 10**, the percentage of threads made by more than 10 comments;
  - **% of thread where user has last word**, the percentage of threads where the leaf node is a comment written by the work author;
  - **Average % of author replies in threads**, the average percentage of comments written by the author in a thread;
  - **Average % of author replies in threads longer than 3**, the average percentage of comments written by the author in threads longer than 3;
  - **Average % of author replies in threads longer than 5**, the average percentage of comments written by the author in threads longer than 5;
  - **Average % of author replies in threads longer than 10**, the average percentage of comments written by the author in threads longer than 10;

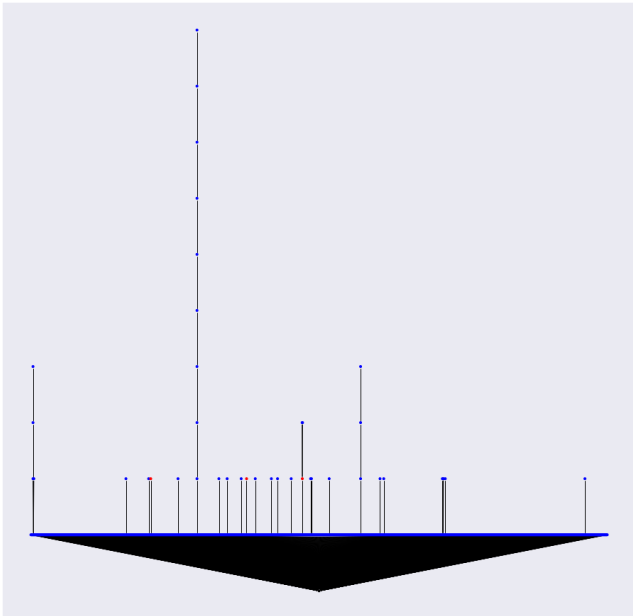
4.3 Works analysis

We can finally proceed with the actual analysis. In particular, we have chosen to show the analysis of two works, taken from two different communities and with different characteristics, so that we can compare them and have an idea of how our metrics model the features of a work.

To compare these works in a more efficient way, let us show the two visualisations side by side and put the result table just below them.



(a) Everyone in Town's Mighty Scared, What With All the Gold and the Outlaws Marvel, ID=12037497



(b) The Least of All Possible Mistakes Sherlock Holmes, ID=330685

Figure 4.4: Analysed works

We can see how the works are pretty different even just taking a glance at the pictures. *Work one*, in Figure 4.4a is more articulated, branched and coloured in red than *work two*, in Figure 4.4b.

Tree Overview, Author Role and Commenters Role

Metrics	**work 1**	**work 2**
Number of comments	200	1438
Tree depth	6	10
Tree breadth	93	1400
Widest layer	1	1
% of single comments	6%	95.34%
Is Author the most active user?	yes	no
% of Author comments	43.50%	0.208%
% of Author-replied comments	76.99%	0.209%
Number of distinct users	34	886
Average number of comments per user	5.88	1.62
Average number of comments per user (without user)	5.88	1.62

Table 4.1: Tree Overview, Author and Commenters Role Table

Let us take a look at Table 4.1.

Work 2 has much more comments than work 1, and hence as you may expect it is deeper and wider; the widest layer, however, is the first one in both cases, which how we can see later is a general trend.

The percentage of single comments could in some way give us an incipit for the metrics about the author. As shown in the table, only 6% of comments in work 1 are directly to the work itself, while all the others are replies to other comments and form a thread web; on the other hand, work 2 has an incredibly high percentage of single comments, more than 95%. Our hypothesis is that the author could play a fundamental role in the construction of threads, replying with comments that become hook points for others and help creating interactions. The huge breadth in work 2 could then be

correlated to the incredibly low percentage of author comments, as a sign that the author is a key in keeping alive the conversations under the work.

The following metrics seem to agree: the author is the most active user in work 1 but not in work 2; hence, the percentage of author comments (and clearly of author-replied comments) is a really tiny number in work 2 whereas it hits 43% in work 1; the average number of comments per user in work 1 is around four times the 1 in work 2.

Threads

Metrics	work 1	work 2
Average thread length	2.67	2.48
% of threads long 1 or 2	72.04%	99.64%
% of threads longer than 3	27.95%	0.35%
% of threads longer than 5	7.52%	0.07%
% of threads longer than 10	0%	0.07%
% of threads where user has the last word	83.95%	6.89%
Average % of author comments in a thread	45.74%	5.74%
Avg % of author comments in threads longer than 3	40.57%	13.33%
Avg % of author comments in threads longer than 5	40%	0%
Avg % of author comments in threads longer than 10	0%	0%

Table 4.2: Threads Table

Let us move onto thread analysis, discussed in Table 4.2.

The average thread length gives an unexpected result because the two values are not so far from each other, being 2.67 for work 1 and 2.48 for work 2. First things first, let us keep in mind that those metrics are calculated on threads, so it does not matter whether a work has a very high percentage of single comments and the other one is full of threads: they get calculated on the appearing threads.

Although in work 1 the thread web is more dense, these threads are all small and never go above depth 6. On the other hand, work 2 has one really long thread with depth 10 that boosts the mean up a little bit.

The percentage of threads long 2 summed with the percentage of single comments gives a value of 99.64% in work 2, while barely reaches the 72% in the first one. We can then see the percentage of threads longer than 3, 5 and 10. It may be useful to use these metrics along with the last three, which report an average percentage of author comments in threads longer than 3, 5 and 10. Of course, the values decrease as the thread length increase, but in some way they give us an idea of how the author is involved in the discussions: if authors get involved in long conversations with their users, it might suggest that they care about readers' opinions and appreciate criticisms and are happy to commit in long threads.

4.4 Works analysis aggregated per community

In this final section we are interested in elaborating some thoughts about the big picture.

Clearly, comparing single works cannot give us an indicative idea about trends and patterns, because they could all be specific cases. Thus, we are now going to aggregate the metrics we have seen before into new ones, in order to perform a community-wise analysis.

Let us define some new metrics which will come from the ones we already defined:

- **% Author is the most active user**, the percentage of works where the author is the most active user;
- **% single comments are dominant**, the percentage of works where single comments are more than thread-originating ones;
- **Avg % author comments**, the average percentage of author comments among all works;

- **Avg % single comments**, the average percentage of single comments among all works;
- **Avg number of comments per user**, the average number of comments per user, considering the author in the counting;
- **Avg thread length**, the average length of a thread;
- **Avg % threads longer than 3**, the average percentage of threads longer than 3;
- **Avg % threads longer than 5**, the average percentage of threads longer than 5;
- **Avg % author comments in threads longer than 3**, the average percentage of author comments in threads longer than 3;
- **Avg % author comments in threads longer than 5**, the average percentage of author comments in threads longer than 5;
- **Avg % author comments in threads longer than 10**, the average percentage of author comments in threads longer than 10;

Let us now proceed to take a look at the aggregated tables below.

Overview					
Community	% where Author is the most active user	% where single comments are dominant	Avg % author comments	Avg % single comments	Avg number of comments per user
Marvel	86.10%	21.26%	34.89%	25.66%	2.63
Harry Potter	78.96%	30.79%	30.87%	33.65%	2.69
Sherlock Holmes	90.48%	15.59%	37.62%	20.28%	2.90
The Lord of the Rings	85.11%	18.31%	36.04%	23.08%	3.66
Percy Jackson	80.97%	24.85%	31.96%	28.68%	2.68
Twilight	65.26%	44.12%	25.82%	44.00%	3.22
Warriors	70.21%	25.00%	31.14%	28.89%	6.22

Table 4.3: General Overview Table

As we did before, we first have an overview with Table 4.3, containing all the information but the thread-related for the sake of clarity.

Generally speaking, there seem to be some trends which hold for all the analysed communities, with a slight variance among them that could be influenced by several factors, e.g. its size in term of works, the amount of users participating, the average length of the content, the age range of users and so on; analysing this sort of factors could become a topic on its own and it would be interesting to investigate for correlations.

In more than 70% of works, with a slightly smaller value for Warriors, the author is the most active user, suggesting that actively participating in the community that develops under a work is common behaviour for authors. In the previous section, we made the guess that the activity of the author in a community produces a more dense web of threads, encouraging users to dialog and post comments; this seems to be confirmed by the next column, where we can observe that the percentage of single comments is pretty low and hence that threads are generally more common: this time, we have values on a range of 15 to 44, but with a mean of about 25.

It is interesting to note how the average percentage of author comments is pretty much the same among all communities. The mean is 32%, meaning that in every work there is a third of comments made by the person who wrote that work; it is not a huge number, but still it is notable since it is achieved by only one person.

Finally, there is the number of comments posted by users on average, and the values are really similar in this case as well, staying between 2.6 and 6.2. Actually, 6.22 seems to be a bit higher than the mean, and this is an issue we get also in the next table for the Warriors fandom. The reason behind this may be found in those anomalous works we have seen before: Warriors is the smallest community and has at least a couple of biasing works, that might push the values in a not totally accurate direction.

General Threads Overview						
Community	Avg thread length	Avg % threads longer than 3	Avg % threads longer than 5	Avg % author comments in threads longer than 3	Avg % author comments in threads longer than 5	Avg % author comments in threads longer than 10
Marvel	2.23	8.46%	1.86%	37.39%	41.30%	44.99%
Harry Potter	2.19	7.27%	1.47%	36.37%	40.32%	44.78%
Sherlock Holmes	2.27	10.47%	2.37%	38.48%	41.98%	46.16%
The Lord of the Rings	2.37	11.64%	3.06%	38.68%	42.12%	46.66%
Percy Jackson	2.36	10.99%	2.66%	36.29%	40.21%	44.23%
Twilight	2.24	6.65%	1.58%	36.86%	42.03%	47.14%
Warriors	5.27	16.29%	6.58%	35.78%	38.49%	40.04%

Table 4.4: General Threads Overview Table

Let us now analyse the last set of metrics, the one regarding threads.

Similarly to previous one, in Table 4.4 we have general trends as well, this time even more close to each other. The average length of a thread is around 2.3 comments, with a peak of 5.27 for Warriors community probably due to the bias as we said before; hence, it seems like the majority of the threads are not particularly extended, but instead are made of only few comments. To find support for this, we can just take a look at the following two columns, containing respectively the average percentage of threads longer than 3 and 5. The numbers here are pretty small, never getting over 16% and 7%, respectively, and with values even smaller if we exclude Warriors from the counting.

The last three columns are probably the most interesting, but before looking at them we need to remember that these data are calculated only on actual threads longer than some value; this means that we have to be careful on how we read them, because we are still talking about an incredibly small percentage of threads.

That being said, Table 4.4 shows the average percentage of author-made comments in threads longer than 3, 5 and 10 is in increasing order: the longer the threads are, the more prone to comment the author becomes, and this once again tells us how authors are generally deeply involved in their own works. We are going to analyse a bit more in detail this aspect in the next chapter, but it still is an interesting outcome.

5 Role of the user in different contexts

This chapter goes through the process of analysing and comparing users metrics when taken in different contexts. The goal we are pursuing this time is to understand if there is a common way of behaving for users in these kind of networks or if the role of a user with respect to the work drastically changes it.

5.1 Building the contributions dataframe

The first step to compute is to build a Pandas DataFrame containing some useful information. Our goal is to have an idea on how the way users write comments change whether they are under their own works or under others'. To achieve this, we are going to look for the amount of comments posted and works written and to inspect centrality metrics of specific users in the two different contexts, i.e. for own and others' works.

5.1.1 Assumption on users

Before starting to build this DataFrame, we would like to know on how many communities is an user active, on average. The reason is that we want to understand if it is safe to assume a user writes mostly on one single fandom or if we should consider others too; if users usually write comments under more than one community, I would have to compare for each user not only the activity on different contexts but also on different fandoms, increasing the complexity.

Luckily, while performing some tests it seemed like a user is only active on one fandom, at least for the communities we are analysing: a user who wrote works on Marvel will stay on Marvel and will not be part of the other six communities. This result may be just a fortunate case: we expect it to be different if run on the entire set of communities.

Anyway, we decided to proceed assuming what said above; we simply iterate on each user for each fandom, and analyse written works and written comments by that specific user in that specific community. If a user were to be active in more than one, then it would just appear in more than one analysis.

5.1.2 Subset of users

Not every user could be relevant to our study. In fact, there is a pretty great number of users who did not write any work or even any comment. Furthermore, considering the entire set of users of each community would be exceptionally costly.

To overcome this issue, we performed our analysis on what it would seem to be a much smaller set of users: we picked only users who wrote more than ten works, resulting in a total of almost 24000 users among all communities. Table 5.1 below shows some statistics to have an idea of the amount of data we are dealing with:

Community	Total users	Authors (wrote at least a work)	Authors who wrote more than 5 works	Authors who wrote more than 10 works	Avg number of works per author
Marvel	448720	18.70%	19.55%	10.36%	5.30
Harry Potter	383228	17.38%	13.96%	6.98%	4.20
Sherlock Holmes	130183	18.74%	17.34%	9.17%	5.09
Lord Of The Rings	38801	17.70%	10.76%	5.58%	3.52
Percy Jackson	48590	15.52%	8.07%	3.14%	2.55
Twilight	27584	17.81%	6.96%	2.58%	2.23
Warriors	4112	34.55%	4.99%	1.90%	1.94

Table 5.1: Statistics about authors

Just to make it clear, while the third column is computed over the total number of users in a community, the following three are computed over the total number of authors in that community. As we can see, numbers tend to be pretty high for fandoms like Marvel and Harry Potter, counting together about 830000 users, of which only around 145000 are authors; still, computing metrics for all the authors would be dramatically slow, and hence we found a convenient trade-off looking only at those who wrote more than five works, raising this threshold to ten for Marvel because the workload was too intense to handle for the machine in a decent time.

The last column shows the average number of works written per author. It is a fundamental piece, because they will be the first works we will iterate on, as we are shortly going to see.

5.1.3 User behaviour metrics

Let us define the metrics we are going to use to build our DatFrame, in which each row will represent one user of one community.

- **Number of comments** posted, under *own* works and *others'* works;
- **Number of works commented**;
- **Number of works written**;
- **Average number of comments** posted, under *own* works and *others'* works;
- **Average PageRank rank** on *own* works and *others'* works;
- **Average in-degree** on *own* works and *others'* works;
- **Average out-degree** on *own* works and *others'* works;

The first four metrics can be easily gathered just by looking at the comments DataFrame. The others can be instead retrieved by the user-user graphs analysis we performed earlier in Chapter 3.

5.1.4 Building process

Let us now dive into the actual process of DataFrame construction. For each fandom, we iterate on all users who have written more than ten works. We gather some basic information, such as the number of works written and commented; in fact, what we are interested in is one list of the works written and one of the works in which they have posted at least one comment.

At this point, our analysis splits in two parts. The first one involves the works written by the user. We iterate on each of these works, sum up the number of author-made comments and retrieve centrality metrics of the author on that specific work. The procedure is then repeated, this time iterating on works in which the user posted one or more comments.

There may be some cases in which centrality metrics are null, mostly when the user in playing the role of the commenter; to overcome this, we just set those values to null, because it means that the user had no relevance in the user-user graph.

After computing all these values for a user, we put them together in a DataFrame averaging out centrality metrics in order to have a general idea of the user relevance. An interesting thing at this point would have been to compare the average relevance on this local community made of written and commented works to the actual relevance of the same user in the entire community. Moreover, there could have been more factors to take into account to analyse this apparent change of behaviour, for instance the participation of the user in long threads in the two cases; unfortunately, for a matter of time we are only focusing on above said metrics.

Once the procedure is done for each user in each community, we have a pretty large DataFrame we are now going to analyse.

5.2 Analysing contributions DataFrame

Let us now analyse a couple of rows from the DataFrame, just to understand how it is built. After that, we are going to see some statistics about it and, finally, we will take a look at the aggregated analysis per community.

Comparing two users metrics

Let us take two randomly-chosen users, from two different communities, with a huge difference in metrics. The DataFrame snippet is shown below in Table 5.2:

User	Comments posted		Works		Avg number of comments posted	
	own works	others' works	written	commented	own works	others' works
Elenduen [Marvel]	4884	5144	19	83	257.05	61.97
solangelo_percabeth [Percy Jackson]	15	15	6	3	2.5	5

Table 5.2: Contributions - works and comments

We can see two users: the first one from Marvel and the second one from Percy Jackson. There is a huge difference in the amount of comments posted and also in works written and commented. While the first user has about 5000 comments posted on their own works as well as in others', the second one only has fifteen in both cases. As said before, the values for works metrics also change: the first user is pretty prolific, with 19 works written and 83 commented, while the second one only wrote 6 and commented 3. The following metrics are derived from the previous ones, and represent the average number of comments posted, calculated as comments posted over works written or commented.

User	Avg PageRank rank		Avg out-degree		Avg in-degree	
	own works	others' works	own works	others' works	own works	others' works
Elenduen [Marvel]	83	62.52	80	13	80	13
solangelo_percabeth [Percy Jackson]	11	1.37	6	0	10	1

Table 5.3: Contributions - user centrality metrics

Moving onto Table 5.3, we can see as expected that the values for first user are bigger than those of the second one. However, there is an hint for next analysis: values for others' works are smaller than for own works, and that occurs in both users. A part from the first user's PageRank rank, other differences are pretty relevant, with values from six to ten times bigger when the user is author.

As we said in the previous chapter, comparing two random users is pretty irrelevant and cannot tell us anything. Our next step is thus to analyse the big picture and see what we can discover.

5.3 Aggregated analysis per community

Let us finally take a look at the final contributions table.

Fandom	Total users	% Analysed users	% Authors who commented own works	% Authors who commented others' works	% Authors who wrote no comments
Marvel	448720	1.93%	94.25%	96.89%	3.10%
Harry Potter	383228	2.42%	84.23%	90.54%	9.45%
Sherlock Holmes	130183	3.25%	88.67%	94.04%	5.95%
The Lord of the Rings	38801	1.90%	82.70%	87.43%	12.56%
Percy Jackson	48590	1.25%	89.32%	93.59%	6.40%
Twilight	27584	1.23%	73.39%	77.48%	22.51%
Warriors	4112	1.72%	78.87%	83.09%	16.90%

Table 5.4: Aggregated Contributions Table - general

Above in Table 5.4 we can see some generic data on authors. First of all, there is the total numbers of users and the percentage of them who respect the condition defined, i.e. being an author and having written at least five (or ten for Marvel) works; as we could imagine, the percentage over the total is pretty small.

Starting from this set of authors, we analysed how many of them commented their own works, how many commented others' and how many instead did not post any comment at all. This last number is relatively small, suggesting that users who write works tend to feel and be more part of the community, be it their works' or others'. The other two, on the contrary, are impressively high, reaching peaks of more than 90% and with the lowest value being 73%: more than seven authors out of ten use to comment both under their works and other people's.

Fandom	% of Authors Avg Comments (own) > Avg Comments (others')	% of Authors PageRank rank (own) > PageRank rank (others')	% of Authors Out-degree (own) > Out-degree (others')	% of Authors In-degree (own) > In-degree (others')
Marvel	48.51%	54.05%	69.86%	72.19%
Harry Potter	32.65%	38.74%	52.84%	55.76%
Sherlock Holmes	38.76%	37.10%	54.62%	56.51%
The Lord of the Rings	25.54%	43.37%	44.59%	45.27%
Percy Jackson	26.92%	50.24%	51.55%	53.53%
Twilight	9.64%	42.69%	38.01%	43.27%
Warriors	18.30%	33.80%	35.21%	35.21%

Table 5.5: Aggregated Contributions Table - comments and user-user metrics

We can now focus on Table 5.3 and take a look at the general trends for comments and user-user

metrics, investigating a little bit more. As it may be reasonable to think, the number of authors who left more comments under their works than under others' is not huge: in fact, it does not even reach the 50%. It is not a surprise: even though authors are very talkative under their works, care about their readers' opinions and stimulate conversations, it would be quite improbable to see a user writing on average more comments on their own works; this is mostly due to quantity reasons, i.e. the amount of works a person writes will statistically be smaller than the one they can read, even just for a matter of time.

For what concerns user-user metrics, the pattern seems to be that PageRank rank is always smaller than Out-Degree, which is always smaller in turn than In-Degree; this, however, does not hold for Twilight, in which Out-Degree is slightly tinier than PageRank rank. Since PageRank is correlated to in and out degree of a node, and ignoring Twilight's small difference, this pattern makes sense.

Generally, it seems more frequent for the in-degree to be higher in own works than the out-degree, and that could mean that comments written under own works are more prone to be replied to by other users.

Conclusions

Although the analysis brought to light very interesting results, there is no doubt that the research on only seven communities of one single platform can only be a starting point for more measures to make. Indeed these seven communities were chosen to be different in their characteristics so that we could generalise; however, we have seen how easy it is to find works with biasing features - just think about anomalous works or orphan comments - and how difficult it is to find a trade-off between computability - in terms of time and processing power - and fair measures.

In Chapter 3 we investigated the relationships between users: how much they conversate, with how many people and we studied some metrics on them. We discovered that there is a frequent star topology, with the author in the middle as the most relevant node. We have seen how different works may have different clustering coefficient and reciprocity values, and we discovered that the clustering coefficient tends to be low, especially on works with lots of comments, while reciprocity does not seem to depend on analysed factors; there seem to be an inverse correlation between the clustering coefficient and the number of comments, while the reciprocity seem to distribute evenly, regardless of the number of comments. We have also analysed the users in a graph on a centrality level preparing the ground for next analysis.

In Chapter 4 we moved our attention towards the structure lying underneath the whole comments section of a work. Visualising it as a tree helped us understand more in detail the way users interact, realising for instance that authors play an important role in the local community built around their works: the more the author commented, the more the threads would grow deeper, encouraging dialogue and interactions. We had to face the problem of orphan comments and anomalous works, that forced us to deal with the concept of bias. We understood that there are some general trends, valid among all communities, in some more than in others, and hence that marginal factors such as fandom size, type of content and consumers' age are less relevant than we thought.

In Chapter 5 we deepened our research on the author, as we earlier found has a great influence on the interactions between users. It turns out that the majority of authors are also active users, and hence are used to post comments under their own and others' works. We also found out that there is a distinct percentage of users who tend to over participate in the comments section of their own works, i.e. to be far more active there than when they are simply reading other people's works; this behaviour, though, is not the majority. It is relevant to note that we may be just scratching the surface with our measures, probably because of thresholds we set, and hence a more sophisticated analysis could be more accurate and reveal different patterns.